Gruppe 3: Dinh Xuan Truong Ung, Roman Pauli, Amine Rekik, Daniel Brenner

Analyseaufgabe 2: Aufruf kritische Methoden

1. Annahmen und Voraussetzungen:
    - Beim erstellen ein Opal-Projekt benötigen wir eine gültige jar-Datei (Test.jar), die die kompilierten Klassen des Java-Programms enthält
    - Fehlermeldungen: Es wurde keine File gefunden => IllegalStateException() wird geworfen und es wurde eine ungültige File gefunden => IllegalStateException() wird geworfen

2. Konkrete Nutzung: Ausführen der Analyse:
    - Eingabe: Der erste Parameter ist der Pfad zur .jar-Datei oder einem anderen Bytecode-Container, der analysiert werden soll. Weitere Parameter sind die sicherheitskritischen Methoden, nach denen gesucht werden soll, im Format <Klassenname>.<Methodenname>. Beispiel Befehl: run System.getSecurityManager (Beispiel: SecurityMethodAnalyzer.jar /path/to/Test.jar System.getSecurityManager System.setSecurityManager)
    - Ablauf:
        1. Einlesen der Eingabeparameter und Ausgeben der zu suchenden Methoden als ListBuffer[String] zurück
           val searchedMethod: ListBuffer[String] = readInput(args)
           val filePath: String = searchedMethod.head
           searchedMethod.remove(0)
        2. OPAL-Projekt erstellen, einschließlich Fehlermeldungen
           val project = createProject(new java.io.File(filePath))
        3. Durchsuchen jede Anweisung(instruction) der Klassen und Methoden, um sicherheitskritische Methodenaufruf zu identifizieren. Mithilfe der MethodInvocationInstruction prüft der Code ob die aufgerufene Methode eine der gesuchten sicherheitskritischen Methoden ist
        4. Prüfen auf sicherheitskritische Methodenaufrufe mit methodCalled() im Vergleich mit searchedMethods die Klassen und Methodennamen. Falls ja, gibt methodCalled die Methode als String zurück(zB: System.getSecurityManager). WasFoundInClass wird auf true gesetzt, um später bei startOutput ausgibt. Andernfalls wird ein leerer String zurückgegeben.
        5. Ausgeben der Ergebnisse output druckt die gefundenen Methoden aus, deren wasFoundInClass true sind

3. Konfigurationsmöglichkeit:
    - Erweiterung der gesuchten sicherheitskritischen Methoden: neue zu überprüfene Methoden einfach als zusätlcihe Parameter beim Aufruf des Programms

hinzugefügt (SecurityMethodAnalyzer.jar /path/to/Test.jar System.getSecurityManager System.setSecurityManager COSParser.isCatalog)

- Anpassungsmöglichkeit der Ausgabe einfach die println-Befehl in main-Methode ändern